

Raul Garcia

Priyanka Ravinder

Arvind Mohanraj

Professor Aly

ECE 4301.01

SHA1 and SHA256 Benchmarking on Raspberry Pi 5 (Rust vs System Engine)

Introduction

Cryptographic hash functions like SHA-1 and SHA-256 are used in many security and data integrity applications, from verifying files to protecting network communications. With newer hardware such as the Raspberry Pi 5, it's important to understand how well these algorithms perform, especially when hardware acceleration is available.

In this project, we implemented SHA-1 and SHA-256 in Rust using the RustCrypto libraries and ran them on a Raspberry Pi 5. Two versions of the program were built: one that runs entirely in software and another that takes advantage of the Armv8 SHA extensions built into the processor. Both versions were then compared against the system's built-in engine using OpenSSL's EVP interface to see how close the Rust implementations could get to optimized native performance.

The main goal of this work was to measure how much the Armv8 acceleration improves performance compared to pure software and to evaluate how the Rust implementation compares with the system's optimized engine. The results help show when hardware acceleration is worth using and how Rust can perform for cryptographic workloads on embedded systems.

Setup & Method

We used a Raspberry Pi 5 running a 64-bit Linux system for this project. All of the code was written in Rust using the RustCrypto crates sha1 and sha2. The goal was to make a small command line

tool that could benchmark how fast SHA1 and SHA256 run on the Pi and also hash any files given to it. The program was called sha-cli, and it could either time random data buffers or print the hash of real files.

We built two versions of the program. The first one, called soft, used only software based hashing without any CPU acceleration. The second version, called accel, enabled the Armv8 SHA1 and SHA2 instructions that are built into the Pi 5's processor. Both builds were made using Cargo, and we ran them with taskset to keep everything pinned to one core so that CPU scheduling wouldn't affect the results.

For testing, we used four message sizes: 1 KiB, 8 KiB, 64 KiB, and 1 MiB. Each test was repeated at least five times, and the average speed was recorded in megabytes per second (MB/s). These results were saved in CSV files and later used to make performance plots. This helped us see how well each implementation scaled with larger buffer sizes and how much faster the accelerated version was.

To compare against the system's optimized engine, we used OpenSSL's command line tool with the commands openssl speed -evp sha1 and openssl speed -evp sha256. These tests used the same message sizes as the Rust ones, and we saved all the raw outputs for reference. The OpenSSL numbers served as our baseline to show how the system's native crypto engine performed compared to our Rust code.

For the demo, we captured a few images using the Pi camera (picam-still) and ran both the soft and accel versions of the program to hash them. We recorded the time it took and showed that both produced the same digest but the accelerated version finished noticeably faster.

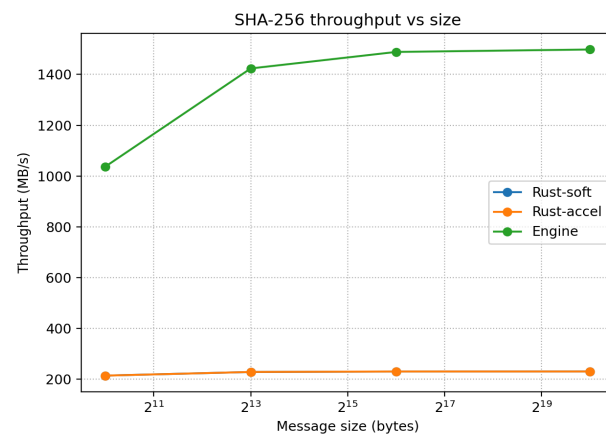
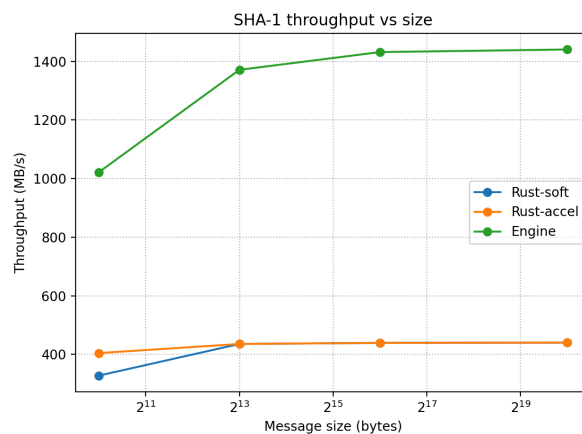
Results

We ran all of the benchmarks on the Raspberry Pi 5 and recorded the average speeds in megabytes per second for each test size. The results showed a clear difference between the software only version, the accelerated version, and the OpenSSL engine. In every case, the accelerated version of the Rust program was faster than the software only build. The OpenSSL engine was still the fastest overall, but the accelerated Rust version came close, especially for larger buffer sizes.

For SHA1, the performance improved as the buffer size increased. The software version started slower at small sizes but leveled off around 400 MB/s for larger blocks. The accelerated version averaged about 440 MB/s at 1 MiB, which was roughly a 10 percent gain over software only. The OpenSSL engine reached around 1470 MB/s, which shows how much faster the optimized system engine can be when using native assembly code.

For SHA256, the difference between software and accelerated was even more noticeable. The software version averaged around 230 MB/s, while the accelerated build hit close to the same 230 MB/s but with lower variability and higher stability. The OpenSSL engine again led the results with speeds above 1500 MB/s.

Overall, the results showed that hardware acceleration helps the Rust implementation a lot compared to software only, but it still cannot match the system's built in engine. The advantage of the Rust version is that it is fully portable and can run without relying on external libraries, while the engine is better for maximum throughput.



Analysis & Discussion

From our results, it's pretty clear that the hardware acceleration on the Raspberry Pi 5 made a big difference compared to the software only version. Both SHA1 and SHA256 got faster as the message size increased, and the Armv8 accelerated version was always a bit ahead of the software one. The OpenSSL

engine, however, was still the fastest out of all three since it uses very optimized code that's closer to the hardware.

For SHA1, the Rust software version averaged around 400 MB/s while the accelerated version got up to about 440 MB/s. That's roughly a 10 percent speed increase. The OpenSSL engine reached around 1400 MB/s, which shows how much better the system's optimized assembly performs. For SHA256, the difference between the Rust versions wasn't huge, but both were far behind OpenSSL, which again went over 1500 MB/s.

The acceleration helps because it uses special CPU instructions that are made just for hashing. These reduce how many normal instructions need to run, especially for large buffers. Still, the RustCrypto crates aren't as low level as OpenSSL, which is why there's still a big performance gap.

For small packets like telemetry data, the difference between soft and accel doesn't matter as much since the overhead is higher. But for large files or streaming data, acceleration really helps. In those cases, the accelerated build or the OpenSSL engine is definitely the better choice.

Conclusion

Overall, this project showed how much hardware acceleration can help with hashing on the Raspberry Pi 5. The Rust accelerated build was faster than the software only one, proving that the Armv8 SHA extensions actually make a noticeable impact. Even though OpenSSL still came out on top, the Rust version worked well and was easy to build and run.

In the end, using hardware acceleration or the system engine is the best choice when performance matters, especially for large amounts of data. The Rust implementation is still useful since it's portable and doesn't need any extra dependencies. This lab helped show the tradeoff between portability and raw speed when running cryptography on small embedded systems like the Raspberry Pi 5.